



Lecture # 15-16 Introduction to SQL Programming Techniques

Rashmi Dutta Baruah

Department of Computer Science & Engineering



भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Technology Guwahati

Guwahati - 781039, INDIA





Outline

- Database Programming
- Embedded SQL
- Functions Calls, SQL/CLI
- Stored Procedures, SQL/PSM
- Summary



Database Programming

- Objective: to access a database from an application program (as opposed to interactive interfaces)
- Why? An interactive interface is convenient but not sufficient; a majority of database operations are made thru application programs (nowadays thru web applications)



Database Programming Approaches

- Embedded commands: database commands are embedded in a general-purpose programming language
- Library of database functions: available to the host language for database calls; known as an *API*
- A brand new, full-fledged language (minimizes impedance mismatch) Eg. Oracle's PL/SQL



Impedance Mismatch

- Incompatibilities between a host programming language and the database model, e.g.,
 - type mismatch and incompatibilities; requires a new binding for each language
 - set vs. record-at-a-time processing
 - need special iterators to loop over query results and manipulate individual values



Steps in Database Programming

1. Client program opens a connection to the database server
2. Client program submits queries to and/or updates the database
3. When database access is no longer needed, client program terminates the connection



Embedded SQL

- Most SQL statements can be embedded in a general-purpose *host* programming language such as COBOL, C, Java
- An embedded SQL statement is distinguished from the host language statements by `EXEC SQL` and a matching `END-EXEC` (or semicolon)
 - *shared variables* (used in both languages) usually prefixed with a colon (:) in SQL



Example: Variable Declaration in Language C

- Variables inside `DECLARE` are shared and can appear (while prefixed by a colon) in SQL statements
- `SQLCODE` is used to communicate errors/exceptions between the database and the program

```
int loop;
```

```
EXEC SQL BEGIN DECLARE SECTION;
```

```
varchar dname[16], fname[16], ...;
```

```
char ssn[10], bdate[11], ...;
```

```
int dno, dnumber, SQLCODE, ...;
```

```
EXEC SQL END DECLARE SECTION;
```




SQL Commands for Connecting to a Database

- Connection (multiple connections are possible but only one is active)

```
CONNECT TO server-name AS connection-name  
AUTHORIZATION user-account-info;
```

- Change from an active connection to another one

```
SET CONNECTION connection-name;
```

- Disconnection

```
DISCONNECT connection-name;
```



Embedded SQL in C

Programming Examples

```
loop = 1;
while (loop) {
    prompt ("Enter SSN: ", ssn);
    EXEC SQL
        select FNAME, LNAME, ADDRESS, SALARY
        into :fname, :lname, :address, :salary
        from EMPLOYEE where SSN == :ssn;
    if (SQLCODE == 0) printf(fname, ...);
    else printf("SSN does not exist: ", ssn);
    prompt("More SSN? (1=yes, 0=no): ", loop);
    END-EXEC
}
```



Embedded SQL in C

Programming Examples

- A *cursor* (iterator) is needed to process multiple tuples
- `FETCH` commands move the cursor to the next tuple
- `CLOSE CURSOR` indicates that the processing of query results has been completed



Example C program segment that uses cursors with embedded SQL for update purposes

```
0) prompt("Enter the Department Name: ", dname) ;
1) EXEC SQL
2)     SELECT Dnumber INTO :dnumber
3)     FROM DEPARTMENT WHERE Dname = :dname ;
4) EXEC SQL DECLARE EMP CURSOR FOR
5)     SELECT Ssn, Fname, Minit, Lname, Salary
6)     FROM EMPLOYEE WHERE Dno = :dnumber
7)     FOR UPDATE OF Salary ;
8) EXEC SQL OPEN EMP ;
9) EXEC SQL FETCH FROM EMP INTO :ssn, :fname, :minit, :lname,
   :salary ;
10) while (SQLCODE == 0) {
11)     printf("Employee name is:", Fname, Minit, Lname) ;
12)     prompt("Enter the raise amount: ", raise) ;
13)     EXEC SQL
14)         UPDATE EMPLOYEE
15)         SET Salary = Salary + :raise
16)         WHERE CURRENT OF EMP ;
17)     EXEC SQL FETCH FROM EMP INTO :ssn, :fname, :minit,
   :lname, :salary ;
18) }
19) EXEC SQL CLOSE EMP ;
```



Dynamic SQL

- Objective: executing new (not previously compiled) SQL statements at run-time
 - a program accepts SQL statements from the keyboard at run-time
 - User Inputs a Query at Runtime, SQL Command is prepared, command is executed



Dynamic SQL: An Example

```
EXEC SQL BEGIN DECLARE SECTION;
```

```
varchar sqlupdatestring[256];
```

```
EXEC SQL END DECLARE SECTION;
```

```
...
```

```
prompt ("Enter update command:", sqlupdatestring);
```

```
EXEC SQL PREPARE sqlcommand FROM :sqlupdatestring;
```

```
EXEC SQL EXECUTE sqlcommand;
```



Embedded SQL in Java

- SQLJ: a standard for embedding SQL in Java
- An SQLJ translator converts SQL statements into Java (to be executed thru the *JDBC* interface)
- Certain classes, e.g., `java.sql` have to be imported



Importing classes needed for including SQLJ in Java programs in Oracle, and establishing a connection and default context.

```
1) import java.sql.* ;
2) import java.io.* ;
3) import sqlj.runtime.* ;
4) import sqlj.runtime.ref.* ;
5) import oracle.sqlj.runtime.* ;
```

Note: SQLJ is primarily designed for working with Oracle databases, and support for MySQL using SQLJ is not natively available. However, JDBC drivers can be used as a bridge

...

```
6) DefaultContext cntxt =
7) oracle.getConnection("<url name>", "<user name>",
"<password>", true) ;
```

```
1) string dname, ssn , fname, fn, lname, ln, bdate,
address ;
```

```
2) char sex, minit, mi ;
3) double salary, sal ;
4) integer dno, dnumber ;
```

Java program variables
used in SQLJ



Embedded SQL in Java: An Example

```
1) ssn = readEntry("Enter a Social Security Number: ");
2) try {
3)     #sql { SELECT Fname, Minit, Lname, Address,
        Salary
4)     INTO :fname, :minit, :lname, :address, :salary
5)     FROM EMPLOYEE WHERE Ssn = :ssn} ;
6) } catch (SQLException se) {
7)     System.out.println("Social Security Number does
not exist: " + ssn) ;
8)     Return ;
9) }
10) System.out.println(fname + " " + minit + " " +
lname + " " + address + " " + salary)
```



Multiple Tuples in SQLJ

- SQLJ supports two types of iterators:
 - *named iterator*: associated with a query result
 - *positional iterator*: lists only attribute types in a query result
- A `FETCH` operation retrieves the next tuple in a query result:

```
fetch iterator-variable into program-variable
```

Example: named iterator

```
0) dname = readEntry("Enter the Department Name: ") ;
1) try {
2)     #sql { SELECT Dnumber INTO :dnumber
3)         FROM DEPARTMENT WHERE Dname = :dname} ;
4) } catch (SQLException se) {
5)     System.out.println("Department does not exist: " + dname);
6)     Return ;
7) }
8) System.out.println("Employee information for Department:
" + dname) ;
9) #sql iterator Emp(String ssn, String fname, String minit,
String lname, double salary) ;
10) Emp e = null ;
11) #sql e = { SELECT ssn, fname, minit, lname, salary
12) FROM EMPLOYEE WHERE Dno = :dnumber} ;
13) while (e.next()) {
14)     System.out.println(e.ssn + " " + e.fname + " " +
        e.minit + " " + e.lname + " " + e.salary) ;
15) } ;
16) e.close() ;
```

← Iterator object e of type Emp is created



Example: positional iterator

```
0)  dname = readEntry("Enter the Department Name: ") ;
1)  try {
2)  #sql { SELECT Dnumber INTO :dnumber
3)  FROM DEPARTMENT WHERE Dname = :dname} ;
4)  } catch (SQLException se) {
5)  System.out.println("Department does not exist: " + dname) ;
6)  Return ;
7)  }
8)  System.out.println("Employee information for Department: " +
dname) ;
9)  #sql iterator Emppos(String, String, String, String, double) ;
10) Emppos e = null ;
11) #sql e = { SELECT ssn, fname, minit, lname, salary
12) FROM EMPLOYEE WHERE Dno = :dnumber} ;
13) #sql { FETCH :e INTO :ssn, :fn, :mi, :ln, :sal} ;
14) while (!e.endFetch()) {
15)     System.out.println(ssn + " " + fn + " " + mi + " " + ln
+ " " + sal) ;
16)     #sql { FETCH :e INTO :ssn, :fn, :mi, :ln, :sal} ;
17) } ;
18) e.close() ;
```



Database Programming with Functional Calls

- Embedded SQL provides static database programming
- API: dynamic database programming with a library of functions
 - advantage: no preprocessor needed (thus more flexible)
 - drawback: SQL syntax checks to be done at run-time



SQL Call Level Interface

- A part of the SQL standard
- Provides easy access to several databases within the same program
- Certain libraries (e.g., `sqlcli.h` for C) have to be installed and available
- SQL statements are dynamically created and passed as string parameters in the calls



Components of SQL/CLI

- *Environment record*: keeps track of database connections
- *Connection record*: keep tracks of info needed for a particular connection
- *Statement record*: keeps track of info needed for one SQL statement
- *Description record*: keeps track of tuples



//Reads a Social Security number of an employee and prints the employee's last name and salary

0) `#include sqlcli.h ;`

← Load SQL/CLI libraries (0)

1) `void printDepartmentEmps() {`

2) `SQLHSTMT stmt1 ;`

3) `SQLHDBC con1 ;`

← Declare record handle variables (2-5)

4) `SQLHENV env1 ;`

5) `SQLRETURN ret1, ret2, ret3, ret4 ;`

6) `ret1 = SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &env1) ;`

7) `if (!ret1) ret2 = SQLAllocHandle(SQL_HANDLE_DBC, env1, &con1) else exit ;`

8) `if (!ret2) ret3 = SQLConnect(con1, "dbs", SQL_NTS, "js", SQL_NTS, "xyz", SQL_NTS) else exit ;`

9) `if (!ret3) ret4 = SQLAllocHandle(SQL_HANDLE_STMT, con1, &stmt1) else exit ;`

↑
Set up environment, connection and statement records (6-9).



```
10) SQLPrepare(stmt1, "select Lname, Salary from EMPLOYEE  
where Dno = ?", SQL_NTS) ;
```

← Prepare a SQL statement (10)

```
11) prompt("Enter the Department Number: ", dno) ;
```

```
12); SQLBindParameter(stmt1, 1, SQL_INTEGER, &dno, 4,  
&fetchlen1)
```

← Bound parameters to program variables (12)

```
13) ret1 = SQLExecute(stmt1) ;
```

← Execute SQL statement (13)

```
14) if (!ret1) {
```

```
15) SQLBindCol(stmt1, 1, SQL_CHAR, &lname, 15, &fetchlen1)  
;
```

```
16) SQLBindCol(stmt1, 2, SQL_FLOAT, &salary, 4, &fetchlen2)  
;
```

```
17) ret2 = SQLFetch(stmt1) ;
```

↑ Bound columns in a query to a C variable (15-16)

```
18) while (!ret2) {
```

```
19)     printf(lname, salary) ;
```

```
20)     ret2 = SQLFetch(stmt1) ;
```

```
21) }
```

← retrieve column values into C variables (17, 20)

```
22) }
```

```
23) }
```



Java Database Connectivity

- JDBC: SQL connection function calls for Java programming
- A Java program with JDBC functions can access any relational DBMS that has a JDBC driver
- JDBC allows a program to connect to several databases (known as *data sources*)



Example: Java program segment with JDBC

```
0) import java.io.* ;
1) import java.sql.*
...
2) class getEmpInfo {
3) public static void main (String args []) throws
   SQLException, IOException {
4) try { Class.forName("oracle.jdbc.driver.OracleDriver")
5) } catch (ClassNotFoundException x) {
6) System.out.println ("Driver could not be loaded") ;
7) }
8) String dbacct, passwd, ssn, lname ;
9) Double salary ;
10) dbacct = readentry("Enter database account:") ;
11) passwd = readentry("Enter password:") ;
12) Connection conn = DriverManager.getConnection
13) ("jdbc:oracle:oci8:" + dbacct + "/" + passwd) ;
```

Import JDBC class libraries

Load JDBC driver (4-7)

Create variables (8-9)

Create connection object (12-13)



Example: Java program segment with JDBC

```
14) String stmt1 = "select lname, salary from EMPLOYEE
where Ssn = ?" ;
15) PreparedStatement p = conn.prepareStatement(stmt1) ;
16) ssn = readentry("Enter a Social Security Number: ") ;
17) p.clearParameters() ;
18) p.setString(1, ssn) ;
19) ResultSet r = p.executeQuery() ;
20) while (r.next()) {
21)     lname = r.getString(1) ;
22)     salary = r.getDouble(2) ;
23)     system.out.println(lname + salary) ;
24) } }
25) }
```

Prepared stmt object (15)

← Set the statement parameters

← Execute query, result in object r of type ResultSet



Steps in JDBC Database Access

1. Import JDBC library (`java.sql.*`)
2. Load JDBC driver:
`Class.forName("oracle.jdbc.driver.OracleDriver")`
3. Define appropriate variables
4. Create a connect object (via `getConnection`)
5. Create a statement object from the `Statement` class:
 1. `PreparedStatement`
 2. `CallableStatement`



Steps in JDBC Database Access (continued)

6. Identify statement parameters (to be designated by question marks)
7. Bound parameters to program variables
8. Execute SQL statement (referenced by an object) via JDBC's `executeQuery`
9. Process query results (returned in an object of type `ResultSet`)
 - `ResultSet` is a 2-dimensional table



Database Stored Procedures

- Persistent procedures/functions (modules) are stored locally and executed by the database server (as opposed to execution by clients)
- Advantages:
 - if the procedure is needed by many applications, it can be invoked by any of them (thus reduce duplications)
 - execution by the server reduces communication costs
 - enhance the modeling power of views



Stored Procedure Constructs

- A stored procedure

```
CREATE PROCEDURE procedure-name (params)
local-declarations
procedure-body;
```

- A stored function

```
CREATE FUNCTION fun-name (params) RETURNS return-type
local-declarations
function-body;
```

- Calling a procedure or function

```
CALL procedure-name/fun-name (arguments);
```




SQL Persistent Stored Modules

- SQL/PSM: part of the SQL standard for writing persistent stored modules
- SQL + stored procedures/functions + additional programming constructs
 - e.g., branching and looping statements
 - enhance the power of SQL



SQL/PSM: An Example

```
CREATE FUNCTION Dept_size (IN deptno INTEGER)
RETURNS VARCHAR[7]
DECLARE Tot_ems INTEGER;

SELECT COUNT (*) INTO Tot_ems
FROM EMPLOYEE WHERE Dno = deptno;

IF Tot_ems > 100 THEN RETURN "HUGE"
ELSEIF Tot_ems > 50 THEN RETURN "LARGE"
ELSEIF Tot_ems > 30 THEN RETURN "MEDIUM"
ELSE RETURN "SMALL"
END IF;
```



Summary

- A database may be accessed via an interactive database
- Most often, however, data in a database is manipulate via application programs
- Several methods of database programming:
 - embedded SQL
 - dynamic SQL
 - stored procedure and function